MoA Prediction

A Multi-Model Ensemble for Mechanism-of-Action Classification

Marco Merlonghi | Project Work in Machine Learning and Data Mining

About the Competition

- **Host:** Laboratory for Innovation Science at Harvard (LISH), with the Broad Institute's Connectivity Map (LINCS);
- **Platform:** Kaggle – `https://kaggle.com/c/lish-moa`;
- **Timeline:** launched and closed in 2020 (final deadline Nov. 30, 2020);
- **Scale:** several thousand participating teams ($\sim$4,300 reported by published write-ups);
- **Data origin:** L1000 assay – a high-throughput assay measuring how human cell lines respond transcriptionally to a chemical or genetic perturbation.

Why it matters

Mapping a compound's gene-expression and cell-viability signature to its mechanism of action accelerates **drug discovery** and **repurposing**, replacing slow, costly wet-lab characterisation with a computational first pass.

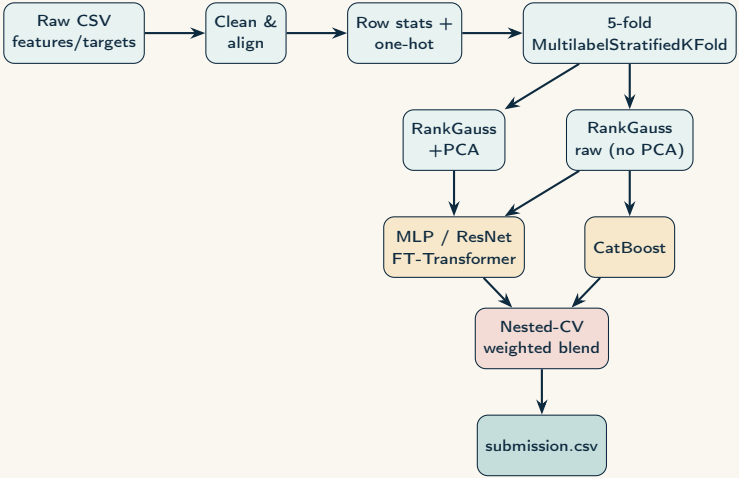
The Problem

- **Task:** multi-label classification of **206 mechanisms of action (MoA)**
- **Input:** gene-expression (`g-`) and cell-viability (`c-`) profiles
- **Metric:** mean multi-label binary log loss, clipped to avoid log(0)
- Control rows (`cp_type == ctl_vehicle`) carry **no MoA** and are excluded from training

Why an ensemble?

Dense nets and tree models fail differently on rare positives $\Rightarrow$ blending several architectures on different preprocessing variants reduces variance.

Pipeline at a Glance



Setup and Data Cleaning

- `iterative-stratification` for multi-label stratified K-fold
- `PYTORCH_CUDA_ALLOC_CONF` to reduce GPU memory fragmentation
- **5 folds**, **3 seeds** $\{42, 202, 2024\} \Rightarrow$ bagging
- AMP enabled only if a GPU is present

```
N_FOLDS = 5
SEEDS = [42, 202, 2024]
PCA_G_COMPONENTS = 50
PCA_C_COMPONENTS = 15
EARLY_STOPPING_PATIENCE = 8
LABEL_SMOOTHING = 0.001
```

- `load_raw_data`: reads `train_features`, `train_targets_scored`, `test_features`, `sample_submission`
- `clean_and_align`: drops NaNs/duplicates, sorts both tables by `sig_id`
- **Strict alignment check** before proceeding:

```
assert features.shape[0] == targets.shape[0]
assert (features["sig_id"].values == targets["sig_id"].values
       ).all()
```

Feature Engineering

Row statistics (separately for `g-` and `c-` blocks):

- sum, mean, std, kurtosis, skewness
- captures the overall activation level per sample

```
df["g_sum"] = df[g_cols].sum(axis=1)
df["g_mean"] = df[g_cols].mean(axis=1)
df["g_std"] = df[g_cols].std(axis=1)
df["g_kurtosis"] = df[g_cols].kurtosis(axis=1)
df["g_skew"] = df[g_cols].skew(axis=1)
# ... mirrored for c_cols
```

Categorical encoding:

- one-hot on `cp_time` and `cp_dose`
- test columns realigned to the training reference columns

```
def encode_categoricals(df, dummy_cols, reference_columns=None):
    """One-hot encodes cp_time/cp_dose, aligned to the training
        columns."""
    df = pd.get_dummies(df, columns=dummy_cols, dtype=np.float64)
    if reference_columns is not None:
        df = df.reindex(columns=reference_columns, fill_value=0)
    return df
```

Four Models, Four Perspectives

MoAMLP
dense, BatchNorm + Dropout

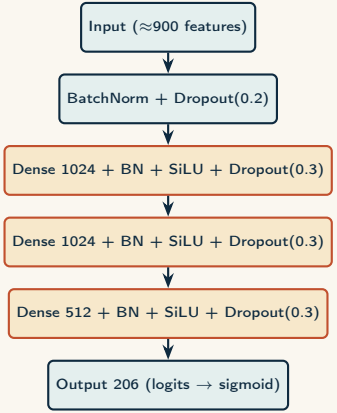
TabularResNet
residual blocks

FT-Transformer
self-attention over features

CatBoost
gradient-boosted trees

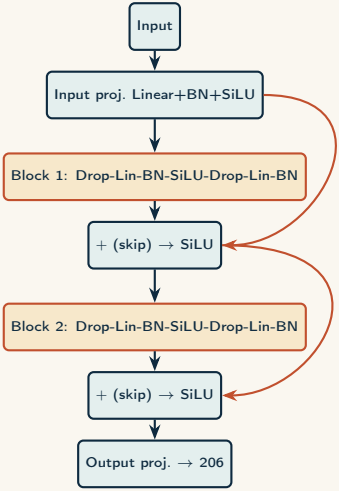
Diversity comes from different *inductive biases*, not just different seeds.

MoAMLP – How It Works



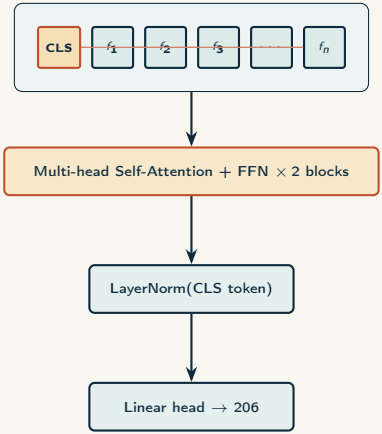
- Pure **feed-forward** network: every neuron combines *all* input features
- BatchNorm stabilises scale, Dropout prevents overfitting
- SiLU (Swish) non-linearity in every block
- Consumes **RankGauss + PCA** features (compact variant)

TabularResNet – How It Works



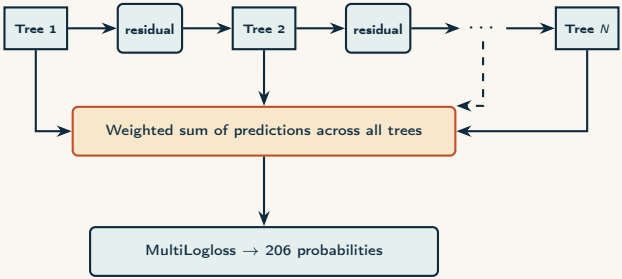
- The curved orange arrows are the **residual (skip) connections**: the block's input jumps straight to the final sum
- Eases gradient flow, stabilises deeper networks
- 2 residual blocks, `hidden_dim = 512`
- Same feature pipeline as the MLP (RankGauss + PCA)

FT-Transformer – How It Works



- Each scalar feature → its own **token** (∼870 tokens/row)
- A learnable **[CLS]** token is prepended
- **Self-attention** (thin lines) connects every token to every other
- The CLS output summarises the row → classification head
- $O(L^2)$ cost ⇒ smaller batch size (64)
- Consumes **raw RankGauss** features (no PCA): one token per original feature

CatBoost – How It Works



- **Gradient boosting**: each tree corrects the residual error of the previous one
- A single multi-output `MultiLogloss` fit, not 206 separate fits
- Splits stay aligned to the original features → trained on **raw RankGauss** (no PCA)
- Targets constant within a fold are excluded from the fit and filled with the observed constant

Metric, Loss Variants and Per-fold Training Routine

Evaluation metric

$$\mathcal{L} = -\frac{1}{N \cdot 206} \sum y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

predictions clipped to $[\epsilon, 1 - \epsilon]$, with $\epsilon = 10^{-15}$.

Optional variants

- **Focal loss:** down-weights easy examples, useful with rare positives
- **Label smoothing** (0.001): $y(1 - \epsilon) + 0.5\epsilon$
- Validation loss **always** uses non-smoothed targets $\rightarrow$ honest comparison

- **AMP** (`torch.amp.GradScaler`) for speed and stability
- **Early stopping** (patience = 8 epochs)
- **ReduceLROnPlateau** scheduler
- **In-RAM checkpointing**: best `state_dict` cloned to CPU, written to disk once at the end
- Explicit `torch.cuda.empty_cache()` so sibling workers reclaim VRAM

Why it matters

Avoids dozens of slow disk writes per fold and prevents GPU memory creep across 45 parallel jobs.

Two Preprocessing Pipelines

PCA variant (MLP, ResNet)

VarianceThreshold → RankGauss (`QuantileTransformer`) → PCA(50) on `g-`, PCA(15) on `c-` → concat + raw dummies

Raw variant (FT-Transformer, CatBoost)

VarianceThreshold → RankGauss, **no PCA** – keeps one token/split per original feature

Both fitted **per fold** on the training split only, cached to disk as `.joblib`, and reused for CatBoost to avoid refitting.

NN Cross-Validation at Scale

- 3 models $\times$ 3 seeds $\times$ 5 folds $=$ **45 jobs**
- `joblib` + `loky` backend, GPUs assigned **round-robin**
- 2 GPUs $\times$ 2 jobs/GPU $=$ **4 parallel workers**
- Predictions averaged over the 3 seeds per (fold, model)

```
NN_JOBS_PER_GPU = 2
N_JOBS_NN = N_GPUS * NN_JOBS_PER_GPU

Parallel(n_jobs=N_JOBS_NN,
       backend="loky")(
 delayed(_train_nn_job)(...)
 for job in jobs
)
```

Ensemble Blending: Methodology

Weight search

- Softmax-reparameterised weights on the simplex
- **Nelder–Mead**, not SLSQP: avoids a zero finite-difference gradient stalling the optimizer at x_0

Nested CV

- Weights fit on $k-1$ folds, scored on the held-out fold
- Prevents an over-optimistic blend score
- Final weights = mean across folds

Two scores reported

naive OOF blend score (same data used to fit the weights) vs. *honest* nested-CV score

Results – Fold-by-Fold Validation Loss

Fold	MLP	ResNet	FT-Transformer	CatBoost
0	0.01666	0.01669	0.01670	0.01707
1	0.01674	0.01685	0.01693	0.01709*
2	0.01632	0.01635	0.01630	0.01676
3	0.01653	0.01655	0.01667	0.01704*
4	0.01680	0.01684	0.01688	0.01704

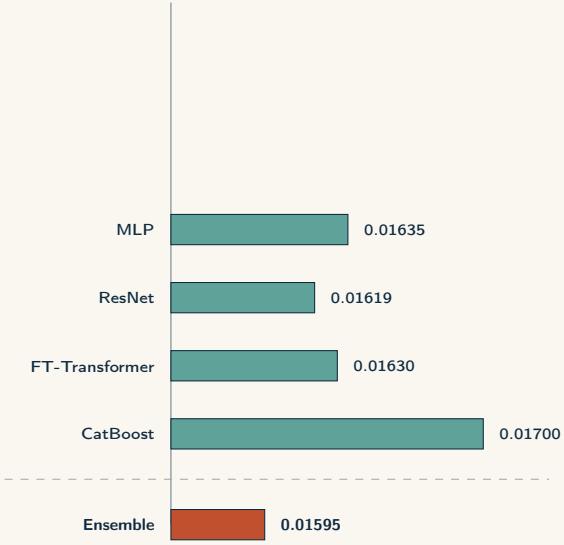
* 1 constant target in the training fold, excluded from the CatBoost fit and filled with the observed constant.

MLP/ResNet/FT-Transformer values are the mean over 3 seeds.
CatBoost is trained once per fold (no seed bagging).

Results – Out-of-Fold Summary

Model	OOF log loss
MLP	0.01635
ResNet	**0.01619**
FT-Transformer	0.01630
CatBoost	0.01700
Ensemble (honest)	**0.01595**

Timing: NN CV 87.0 min | CatBoost CV 28.5 min



Results – Ensemble Blend (Nested CV)

Final score and weights per fold

Fold	Score	w_{mlp}	w_{resnet}	w_{ft}	w_{cb}
0	0.01597	0.061	0.460	0.345	0.134
1	0.01616	0.030	0.475	0.375	0.120
2	0.01555	0.049	0.453	0.357	0.140
3	0.01594	0.042	0.453	0.369	0.136
4	0.01613	0.064	0.455	0.368	0.114

Final weights (mean across folds)

MLP 4.9% | ResNet 45.9% | FT-Transformer 36.3% | CatBoost 12.9%

Honest nested-blend score

0.01595 ± 0.00022

Naive OOF blend (same data used to fit weights): **0.01595** – matches the honest score almost exactly, a sign the blend is not overfit to the validation folds.

Key Engineering Decisions

- **Per-feature RankGauss**, PCA only where it helps (dense nets), never for trees
- **Multi-seed bagging** smooths single-run noise before blending
- **Round-robin GPU scheduling** keeps multi-GPU training balanced
- **In-RAM checkpoints** minimise I/O across 45 parallel jobs
- **Nested-CV blend weights** avoid leaking validation information into the final score